ระบบรวบรวมและเปรียบเทียบราคารถยนต์ Toyota

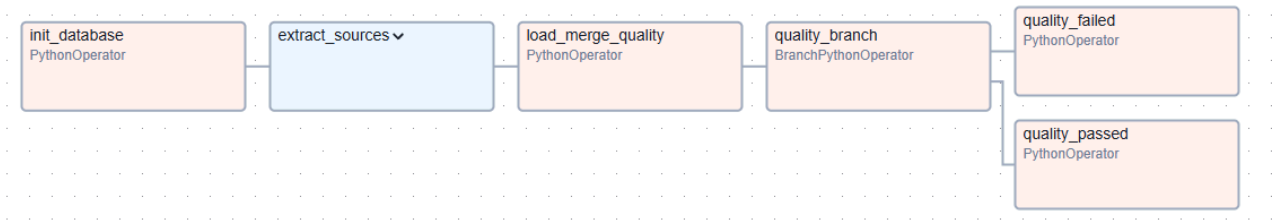
Github Repository: https://github.com/TitaniumSalmon/02739452-22-6621601131

ภาพรวมและผลการดำเนินงาน

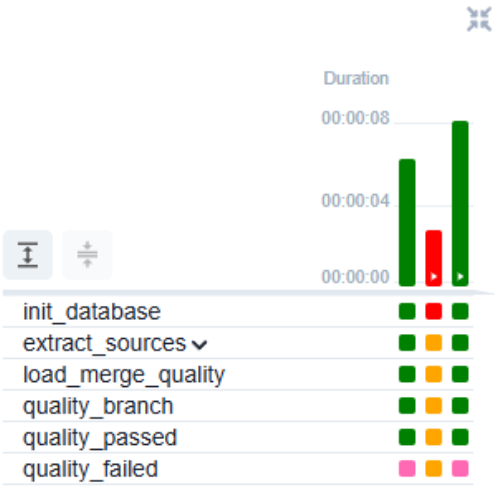
โครงงานนี้รวบรวมราคาของรถยนต์ Toyota จาก 3 แหล่ง ได้แก่ CheckRaka, Toyota นครปฐม และ Toyota JD Cars โดยใช้ Apache Airflow จัดการขั้นตอนดึงและแปลงข้อมูล ก่อนบันทึกลง PostgreSQL และแสดงผลบนเว็บ Dashboard

DAG แยกการดึงข้อมูลแต่ละแหล่งเป็น task ภายใน TaskGroup จากนั้นปรับข้อมูลให้อยู่ในรูปแบบเดียวกัน ตรวจสอบชื่อรุ่นและช่วงราคา และใช้ UPSERT ป้องกันรายการซ้ำเมื่อรันซ้ำ Dashboard อ่านข้อมูลจาก PostgreSQL โดยตรง สามารถกรองตามแหล่ง รุ่นรถ และช่วงราคา พร้อมแสดงกราฟเปรียบเทียบราคาและกราฟจุดที่แสดงข้อมูลรถแต่ละรายการ

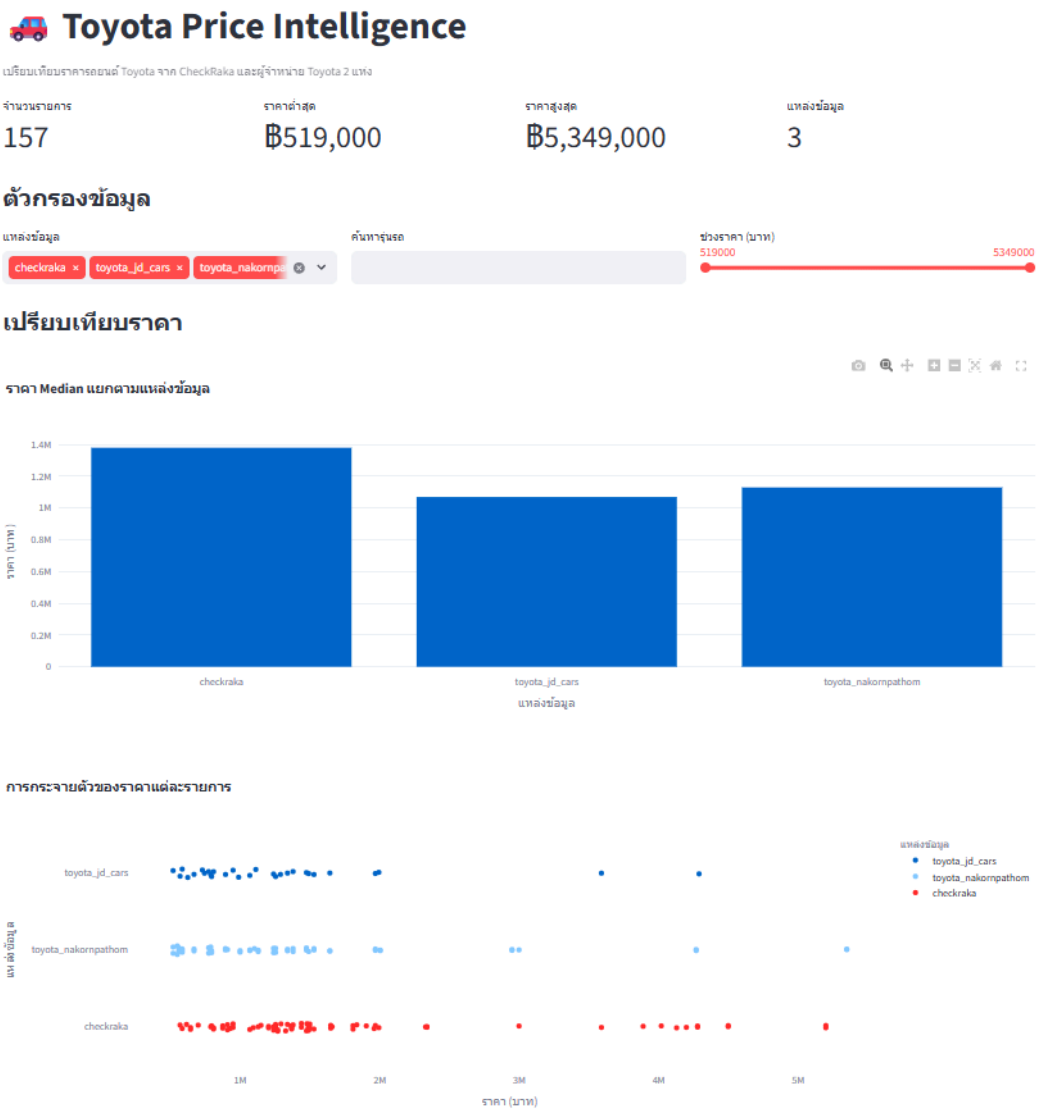
รอบที่ตรวจสอบมีข้อมูลรวม **157 รายการ** ได้แก่ CheckRaka 73 รายการ, Toyota JD Cars 29 รายการ และ Toyota นครปฐม 55 รายการ **หลังตัดซ้ำเหลือ 110 รายการ** และพบรายการที่ตรงกันข้ามแหล่ง 40 กลุ่ม ราคาจากแต่ละเว็บอาจเป็นคนละประเภท เช่น ราคาเริ่มต้นหรือราคาที่ผู้จำหน่ายแสดง จึงเก็บแหล่งที่มาและประเภทราคาไว้ประกอบการเปรียบเทียบ



Graph view ของ DAG



Grid view ของ DAG



ภาพหน้าเว็บ dashboard

ปัญหาที่พบและวิธีแก้

1. **Airflow เริ่มทำงานไม่ได้ในครั้งแรก**

   อาการ: Airflow แจ้งว่าตาราง metadata เช่น log ยังไม่มี และ container ปิดตัว

   การตรวจสอบ: ตรวจ log ของ Airflow และ PostgreSQL พบว่า scheduler กับ webserver เริ่มทำงานพร้อมกับขั้นตอน migration ฐานข้อมูล

   วิธีแก้: ปรับคำสั่ง startup ให้ migration และการสร้างผู้ใช้เสร็จก่อน แล้วจึงเริ่ม scheduler กับ webserver ผลหลังแก้คือ Airflow เปิดใช้งานได้และแสดง DAG toyota_price_pipeline

2. **กราฟจุดแสดง TypeError**

   อาการ: Dashboard แจ้งว่า strip() got an unexpected keyword argument 'jitter'

   การตรวจสอบ: ตรวจข้อความ error พบว่า Plotly เวอร์ชันที่ใช้อยู่ไม่รองรับพารามิเตอร์ jitter ใน px.strip

   วิธีแก้: นำพารามิเตอร์ดังกล่าวออก แล้ว build และเริ่ม Dashboard ใหม่

3. **ไม่พบ robots.txt ของเว็บไซต์ Toyota Thailand**

   อาการ: URL robots.txt ตอบกลับ 404

   วิธีแก้: เปลี่ยนแหล่งข้อมูลดังกล่าวเป็น Toyota JD Cars และตรวจ robots.txt ของแหล่งใหม่นี้ก่อนนำมาใช้

การวางแผนและจัดลำดับงาน

จัดลำดับงานโดยเริ่มจากเลือกหัวข้อและตรวจแหล่งข้อมูล จากนั้นทำการดึงและแปลงข้อมูล จัดเก็บใน PostgreSQL ทำ Dashboard และตรวจการทำงานของระบบ ก่อนเตรียมภาพและเอกสารสรุป หากมีเวลาจำกัด จะให้ความสำคัญกับการทำให้ pipeline และ Dashboard ใช้งานได้ก่อน แล้วจึงปรับหน้าตาและเพิ่มรายละเอียด

ขอบเขตข้อมูลเน้นชื่อรุ่น ราคา ประเภทของราคา และแหล่งที่มา โดยไม่ได้ขยายไปเก็บข้อมูลสเปกรถ โปรโมชั่น หรือรายละเอียดการผ่อนชำระ เพื่อให้มีเวลาแก้ปัญหาและตรวจสอบผลของระบบ

การแบ่งเวลาตัวอย่างสำหรับสอบ 3 ชั่วโมง

ตรวจหัวข้อและแหล่งข้อมูล 20 นาที · ทำ DAG และดึงข้อมูล 70 นาที · จัดเก็บ PostgreSQL และทำ Dashboard 45 นาที · แก้ปัญหาและตรวจจำนวนข้อมูล 25 นาที · จับภาพและจัดทำ PDF 20 นาที